

DSA Assignment #6: 🌲 (1/3)

2500010774 兰玉琪 数学科学学院

1. 题目

E94.二叉树的中序遍历

dfs, stack, <https://leetcode.cn/problems/binary-tree-inorder-traversal/>

思路：

代码：

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        mylist = []
        def inorder(root):
            if not root:
                return
            inorder(root.left)
            mylist.append(root.val)
            inorder(root.right)
        inorder(root)
        return mylist
```

Status	Language	Runtime	Memory	Note
1 Accepted Aug 17, 2025	Python3	0 ms	17.6 MB	

Accepted 71 / 71 testcases passed

cZ88PGJiHH submitted at Aug 17, 2025 16:29

Runtime

0 ms | Beats 100.00%

Memory

17.56 MB | Beats 100.00%

150%

Testcase Test Result

You must run your code first

E108.将有序数组转换为二叉搜索树

<https://leetcode.cn/problems/convert-sorted-array-to-binary-search-tree/>

思路：

代码：

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sortedArrayToBST(self, nums):
        def construct(i,j):
            if i > j:
                return None
            mid = (i+j)//2
            a = nums[mid]
            node = TreeNode(a)
            node.left = construct(i,mid-1)
            node.right = construct(mid+1,j)
            return node
        return construct(0,len(nums)-1)
```

Status ▾

Language ▾

Runtime

Memory

Notes

1

Accepted
Jan 26, 2026

Python3

3 ms

20.4 MB

← All Submissions

Accepted 31 / 31 testcases passed

cZ88PGJiHH submitted at Jan 26, 2026 18:58

Runtime

3 ms | Beats 50.34%

Memory

20.42 MB | Beats 17.17%

40%

Testcase

Test Result

You must run your code first

M102.二叉树的层序遍历

bfs, <https://leetcode.cn/problems/binary-tree-level-order-traversal/>

思路：

代码：

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        if not root:
            return []
        from collections import deque
        mylist = []
        queue = deque([root])
        while queue:
            lst = []
            n = len(queue)
            for _ in range(n):
                node = queue.popleft()
                lst.append(node.val)
                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)
            mylist.append(lst)
        return mylist

```

Status	Language	Runtime	Memory	Note
2 Accepted Nov 14, 2025	Python3	1 ms	18.6 MB	
1 Accepted Aug 17, 2025	Python3	0 ms	18.4 MB	

← All Submissions

Accepted 35 / 35 testcases passed

cZ88PGJiHH submitted at Aug 17, 2025 18:09

🕒 Runtime

0 ms | Beats **100.00%** 🌿

💾 Memory

18.44 MB | Beats **100.00%** 🌿

100%

☒ Testcase
 ☒ Test Result

You must run your code first

M1123.最深叶节点的最近公共祖先

dfs, <https://leetcode.cn/problems/lowest-common-ancestor-of-deepest-leaves/>

思路:

代码:

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from collections import defaultdict
class Solution:
    def lcaDeepestLeaves(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        max_depth = 0
        mydict = defaultdict(list)
        def dfs(node, depth, path):
            nonlocal max_depth
            max_depth = max(max_depth, depth)
            if not node.left and not node.right:
                if depth == max_depth:
                    mydict[depth].append(path.copy())
                return
            if node.left:
                dfs(node.left, depth+1, path+[node.left])
            if node.right:
                dfs(node.right, depth+1, path+[node.right])
        dfs(root, 0, [root])
        for _ in range(max_depth+1):
            myset = set()
            for path in mydict[max_depth]:
                myset.add(path.pop())
            if len(myset) == 1:
                return next(iter(myset))
```

Problem List < > ↺

Description | Accepted × | Editorial | St <

All Submissions

Accepted 81 / 81 testcases passed

cZ88PGJiHH submitted at Apr 07, 2026 15:49

Runtime

3 ms | Beats 48.43%

Memory

20.39 MB | Beats 5.16%

Code

```
Python3 Auto
23 def dfs(root, path):
24     for _ in range(max_depth+1):
25         myset = set()
26         for path in mydict[max_depth]:
27             myset.add(path.pop())
28         if len(myset) == 1:
29             return next(iter(myset))
30
```

Saved Ln 30, Col 9

Testcase >_ Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

M07161: 森林的带度数层次序列存储

tree, <http://cs101.openjudge.cn/practice/07161/>

思路：

代码

```

n = int(input())
result = []
class TreeNode:
    def __init__(self, val=0):
        self.val = val
        self.children = []
from collections import deque
for _ in range(n):
    mylist = input().split()
    i = 2
    dummy = TreeNode()
    dummy.children = [TreeNode(mylist[0])]
    queue = deque([(dummy.children[0], int(mylist[1]))])
    while queue:
        node, node_num = queue.popleft()
        for _ in range(node_num):
            value = mylist[i]
            node_num = int(mylist[i+1])
            i += 2
            newnode = TreeNode(value)
            queue.append((newnode, node_num))
            node.children.append(newnode)
    def houxu(node):
        if node:
            for newnode in node.children:
                houxu(newnode)
            result.append(node.val)
    houxu(dummy.children[0])
print(*result)

```

状态: **Accepted**

Source Code

```
n = int(input())
result = []
class TreeNode:
    def __init__(self, val=0):
        self.val = val
        self.children = []
from collections import deque
for _ in range(n):
    mylist = input().split()
    i = 2
    dummy = TreeNode()
    dummy.children = [TreeNode(mylist[0])]
    queue = deque([(dummy.children[0], int(mylist[1]))])
    while queue:
        node, node_num = queue.popleft()
```

基本信息

#: 52223199

题目: 07161

提交人:
25n2500010774(sleepy(25n2500010774))

内存: 3656kB

时间: 33ms

语言: Python3

提交时间: 2026-04-07 16:24:36

M27928: 遍历树

adjacency list, dfs, <http://cs101.openjudge.cn/practice/27928/>

思路:

代码


```

import sys
from collections import defaultdict
data = sys.stdin.readlines()
n = int(data[0][:-1])
mydict = defaultdict(list)
jiedianzhi = set()
feigenjiedianzhi = set()
for i in range(1,n+1):
    mylist = data[i][:-1].split()
    jiedianzhi.add(mylist[0])
    for s in mylist[1:]:
        feigenjiedianzhi.add(s)
        mydict[mylist[0]].append(s)
for s in jiedianzhi:
    if s not in feigenjiedianzhi:
        rootzhi = s
        break
class TreeNode:
    def __init__(self,val):
        self.val = val
        self.children = []
mydict1 = {}
def construct(s):
    tree = TreeNode(s)
    mydict1[int(s)] = tree
    tree.children = [construct(t) for t in mydict[s]]
    return tree
def show(s):
    mylist = [int(t.val) for t in s.children] + [int(s.val)]
    for m in sorted(mylist):
        if m != int(s.val):
            show(mydict1[m])
        else:
            print(s.val)
tree = construct(rootzhi)
show(tree)

```

状态: Accepted

Source Code

```
import sys
from collections import defaultdict
data = sys.stdin.readlines()
n = int(data[0][: -1])
mydict = defaultdict(list)
jiedianzhi = set()
feigenjiedianzhi = set()
for i in range(1,n+1):
    mylist = data[i][: -1].split()
    jiedianzhi.add(mylist[0])
    for s in mylist[1:]:
        feigenjiedianzhi.add(s)
        mydict[mylist[0]].append(s)
for s in jiedianzhi:
    if s not in feigenjiedianzhi:
        rootzhi = s
        break
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.children = []
mydict1 = {}
def construct(s):
    tree = TreeNode(s)
    mvdict1[int(s)] = tree
```

基本信息

#: 51697462

题目: 27928

提交人:

25n2500010774(sleepy(25n2500010774))

内存: 3924kB

时间: 24ms

语言: Python3

提交时间: 2026-01-22 14:24:16

2. 学习总结和个人收获

www期中周了，没什么时间了，就是做做讲义和每日选做吧